Guia Completa: Lectura de IMU via BLE

Seeed XIAO nRF52840 Sense → Python

Todo explicado desde cero, sin dar nada por sabido

SensorKit Project

Marzo 2026

Índice

I Fundamentos: todo lo que necesitas saber antes de tocar codigo 3

1. Que es lo que queremos hacer (y por que es complicado) 3

2. El sensor: LSM6DS3TR-C 4
- 2.1. Que es un sensor IMU . . . 4
- 2.2. Ejes X, Y, Z . . . 4
- 2.3. Rangos y precision . . . 4
- 2.4. Registros: como se comunica el sensor . . . 5

3. I^2C: como el microcontrolador habla con el sensor 5
- 3.1. Que es I^2C . . . 5
- 3.2. Master y Slave . . . 6
- 3.3. Direccion 0x6A . . . 6
- 3.4. Como se ve una lectura I^2C por dentro . . . 6

4. Bluetooth Low Energy (BLE): como los datos viajan al ordenador 6
- 4.1. Que es Bluetooth y por que "Low Energy" . . . 6
- 4.2. Roles en BLE: Peripheral y Central . . . 7

5. GATT: la estructura que organiza los datos en BLE 7
- 5.1. Que es GATT y por que existe . . . 7
- 5.2. La jerarquia GATT: Perfil, Servicio, Caracteristica . . . 8
  - 5.2.1. Servidor GATT . . . 8
  - 5.2.2. Servicio (Service) . . . 8
  - 5.2.3. Caracteristica (Characteristic) . . . 9
  - 5.2.4. Descriptor (Descriptor) . . . 9
- 5.3. Conexion BLE paso a paso . . . 10
- 5.4. Connection interval (intervalo de conexion) . . . 10

6. Numeros flotantes y bytes: como viajan los datos 10
- 6.1. El problema: BLE solo envía bytes . . . 10
- 6.2. IEEE 754: como un float se convierte en 4 bytes . . . 11
- 6.3. Little-endian y Big-endian . . . 11
- 6.4. memcpy: copiar bytes crudos en Arduino . . . 11

7. Las librerias: quien hizo que y como funciona por dentro 12
- 7.1. Lado Arduino (firmware de la XIAO) . . . 12

7.1.1. ArduinoBLE . . . 12
7.1.2. Seeed_Arduino_LSM6DS3 . . . 12
7.1.3. Wire . . . 13
7.2. Lado Python (tu ordenador) . . . 13
7.2.1. bleak . . . 13
7.2.2. struct (biblioteca estandar de Python) . . . 14
7.2.3. asyncio (biblioteca estandar de Python) . . . 14

II El codigo explicado linea por linea **14**

8. Codigo Arduino completo **15**
8.1. Linea 1: `#include <ArduinoBLE.h>` . . . 15
8.2. Linea 2: `#include "LSM6DS3.h"` . . . 15
8.3. Linea 3: `#include "Wire.h"` . . . 15
8.4. Linea 4: Crear el objeto IMU . . . 15
8.5. Lineas 5–7: Crear servicio y caracteristicas BLE . . . 16
8.6. Funcion setup(): inicializacion . . . 17
8.7. Funcion loop(): el bucle principal . . . 18

9. Codigo Python completo **20**
9.1. Imports . . . 20
9.2. Constantes . . . 20
9.3. Variables globales . . . 21
9.4. Callbacks (funciones que se llaman automaticamente) . . . 21
9.5. Funcion principal . . . 22
9.6. Punto de entrada . . . 24

III Apendices **25**

10. Extension: guardar datos en CSV **25**

11. Diagnostico y solucion de problemas **26**
11.1. El dispositivo no aparece en el scan . . . 26
11.2. Se conecta pero no llegan datos . . . 26
11.3. Los valores no tienen sentido . . . 26

12. Referencias **26**

Parte I

Fundamentos: todo lo que necesitas saber antes de tocar codigo

1. Que es lo que queremos hacer (y por que es complicado)

Nuestro objetivo final es muy simple de describir: queremos leer numeros de un sensor y verlos en nuestro ordenador. Pero hay un problema: el sensor no esta enchufado al ordenador con un cable. Esta en una plaquita pequeña (la XIAO nRF52840 Sense) que se comunica por Bluetooth.

Esto introduce varias capas de complejidad:

1. El sensor no "habla" Bluetooth. El sensor habla un protocolo llamado I^2C (lo veremos despues). Asi que necesitamos un **traductor**: el microcontrolador nRF52840 que esta en la placa, que lee los datos del sensor por I^2C y los reempaqueta para enviarlos por Bluetooth.
2. Bluetooth no es como un cable USB donde simplemente "fluyen" bytes. Bluetooth Low Energy (BLE) tiene una estructura rigida: los datos se organizan en "servicios" y "caracteristicas" (esto se llama GATT, y lo explicaremos a fondo).
3. Del lado del ordenador, necesitamos un programa que sepa hablar BLE, conectarse al dispositivo, y extraer los datos. Para esto usamos Python con la libreria `bleak`.

El flujo completo es:

```
SENSOR (chip LSM6DS3TR-C)
  "Tengo una aceleracion de 0.02g en X"
       |
       | Protocolo I2C (cables fisicos dentro de la placa)
       v
MICROCONTROLADOR (chip nRF52840)
  "Empaqueto esos datos como bytes y los pongo
   disponibles via Bluetooth"
       |
       | Ondas de radio Bluetooth (2.4 GHz, sin cables)
       v
TU ORDENADOR (Mac/PC/Linux)
  "Me conecto por Bluetooth, leo los bytes,
   los convierto de vuelta a numeros"
       |
       | Libreria bleak + struct de Python
       v
TU SCRIPT PYTHON
  "Ahora tengo ax=0.02, ay=0.01, az=1.00
   y puedo hacer lo que quiera con ellos"
```

Cada una de estas flechas es un mundo. Vamos a explicarlas todas.

2. El sensor: LSM6DS3TR-C

2.1. Que es un sensor IMU

IMU significa **Inertial Measurement Unit** (Unidad de Medicion Inercial). Es un chip que mide movimiento. El LSM6DS3TR-C, fabricado por la empresa **STMicroelectronics**, tiene dos sensores dentro:

- **Acelerometro**: Mide *aceleracion lineal*. Si mueves la placa hacia la derecha, el acelerometro detecta esa aceleracion. Tambien detecta la gravedad terrestre: si la placa esta quieta sobre una mesa, el eje que apunta hacia arriba mide aproximadamente $1g$ (donde $1g = 9.81$ m/s^2). Los otros dos ejes miden aproximadamente $0g$.

  El acelerometro funciona internamente con una masa microscopica suspendida por muelles de silicio. Cuando el chip acelera, la masa se queda atras por inercia y deforma los muelles. La deformacion se mide como un cambio en la capacitancia electrica entre placas microscopicas. Ese cambio de capacitancia se convierte en un voltaje, y luego un conversor analogico-digital (ADC) lo convierte en un numero.

- **Giroscopio**: Mide *velocidad angular* (que tan rapido estas girando). Si giras la placa sobre su eje Z, el giroscopio del eje Z mide esa velocidad en **grados por segundo** (dps). Si la placa esta quieta, todos los ejes miden aproximadamente 0.

  El giroscopio usa el efecto Coriolis: dentro del chip hay una estructura que vibra a alta frecuencia. Cuando el chip gira, la fuerza de Coriolis desvía la vibracion en una direccion perpendicular. Esa desviacion se mide como un cambio de capacitancia, se convierte en voltaje, y luego en un numero.

2.2. Ejes X, Y, Z

Cada sensor mide en **tres ejes perpendiculares**: X, Y, Z. Imagina que pones la placa sobre la mesa:

- **X**: Apunta hacia la derecha de la placa.
- **Y**: Apunta hacia "arriba" de la placa (el lado opuesto al conector USB).
- **Z**: Apunta hacia arriba, perpendicular a la mesa.

En total tenemos 6 valores: a_x, a_y, a_z (acelerometro) y g_x, g_y, g_z (giroscopio). Por eso se dice "6 ejes" o "6-DOF" (6 Degrees of Freedom).

2.3. Rangos y precision

El sensor tiene rangos configurables. Un rango mayor permite medir movimientos mas bruscos, pero con menos precision:

Sensor	**Rangos disponibles**	**Default**
Acelerometro	±2g, ±4g, ±8g, ±16g	±2g
Giroscopio	±125, ±245, ±500, ±1000, ±2000 dps	±245 dps

Internamente, el sensor convierte su medicion a un numero entero de 16 bits (valores de -32768 a $+32767$). Si el rango del acelerometro es ±2g, entonces $+32767$ corresponde a $+2g$ y -32768 corresponde a $-2g$. La resolucion es:

$$\frac{2g}{32768} \approx 0.000061 \text{ g por unidad}$$

La libreria que usamos en Arduino hace esta conversion por nosotros y nos da directamente el valor en g o *dps*.

2.4. Registros: como se comunica el sensor

El sensor LSM6DS3TR-C es, en esencia, una pequeña memoria con "casillas" llamadas **registros**. Cada registro tiene una direccion (un numero hexadecimal) y almacena un byte (8 bits, valor de 0 a 255).

Hay dos tipos de registros:

- **Registros de configuracion**: Tu escribes en ellos para decirle al sensor como comportarse. Por ejemplo, el registro `CTRL1_XL` (direccion `0x10`) controla la frecuencia de muestreo y el rango del acelerometro. Si escribes el byte `0x60` en ese registro, le estas diciendo: "muestrea a 416 Hz con rango ±2g".
- **Registros de datos**: El sensor escribe en ellos los valores medidos. Tu los lees. Los datos del acelerometro eje X estan en los registros `0x28` (byte bajo) y `0x29` (byte alto). Juntos forman un entero de 16 bits.

Registro	Direccion	Contenido
WHO_AM_I	0x0F	Siempre vale 0x6A (identifica al chip)
CTRL1_XL	0x10	Configuracion del acelerometro
CTRL2_G	0x11	Configuracion del giroscopio
OUTX_L_XL	0x28	Acelerometro X, byte bajo
OUTX_H_XL	0x29	Acelerometro X, byte alto
OUTY_L_XL	0x2A	Acelerometro Y, byte bajo
...	...	...
OUTX_L_G	0x22	Giroscopio X, byte bajo
OUTX_H_G	0x23	Giroscopio X, byte alto
...	...	...

Pero nosotros no leemos estos registros directamente en nuestro codigo. La libreria `Seeed_Arduino_LSM6DS3` lo hace por nosotros. Cuando llamamos a `myIMU.readFloatAccelX()`, la libreria:

1. Lee los registros `0x28` y `0x29`.
2. Combina los dos bytes en un entero de 16 bits.
3. Multiplica por el factor de escala (0.000061 para ±2g).
4. Devuelve el resultado como un `float` en unidades de g.

3. I²C: como el microcontrolador habla con el sensor

3.1. Que es I²C

I²C (pronunciado "i-cuadrado-ce" o "i-dos-ce") significa **Inter-Integrated Circuit**. Es un protocolo de comunicacion que permite a chips electronicos hablar entre si usando solo **dos cables**:

- **SDA** (Serial Data): Por aqui viajan los datos en ambas direcciones (el micro le envia comandos al sensor, el sensor le devuelve los datos al micro).
- **SCL** (Serial Clock): Es una senal de reloj que marca el ritmo. Cada "tick" del reloj significa "ahora lee un bit de SDA". Esto sincroniza a ambos chips para que no se pierdan datos.

3.2. Master y Slave

En I^2C siempre hay un **master** (quien manda) y uno o mas **slaves** (quienes obedecen):

- **Master**: El microcontrolador nRF52840. El inicia todas las comunicaciones. Dice "quiero leer el registro 0x28 del dispositivo 0x6A".
- **Slave**: El sensor LSM6DS3TR-C. Su direccion en el bus es **0x6A**. Cuando el master pide el registro 0x28, el sensor responde con el byte que tiene almacenado ahi.

3.3. Direccion 0x6A

Cada dispositivo I^2C tiene una direccion unica de 7 bits (valores de 0 a 127, o 0x00 a 0x7F en hexadecimal). Esto permite tener hasta 128 dispositivos en el mismo bus (mismos dos cables). El master incluye la direccion en cada comunicacion para que solo el dispositivo correcto responda.

El LSM6DS3TR-C tiene la direccion **0x6A** porque asi lo diseño STMicroelectronics. Viene fija de fabrica (bueno, se puede cambiar a 0x6B moviendo un pin, pero en la XIAO esta soldado a 0x6A).

3.4. Como se ve una lectura I^2C por dentro

Cuando el micro quiere leer el eje X del acelerometro, la conversacion en el bus I^2C es algo asi:

1. El master envia una condicion de START (baja SDA mientras SCL esta alto — esta es una senal electrica especial que todos los slaves reconocen como "atencion, empieza una comunicacion").
2. El master envia la direccion 0x6A + bit de escritura. El slave responde con ACK ("estoy aqui").
3. El master envia 0x28 ("quiero empezar a leer desde el registro 0x28").
4. El master envia otra condicion de START (repeated start).
5. El master envia la direccion 0x6A + bit de lectura.
6. El slave envia el contenido del registro 0x28 (byte bajo del acelerometro X).
7. El master envia ACK ("dame mas").
8. El slave envia el contenido de 0x29 (byte alto del acelerometro X).
9. El master envia NACK ("ya no quiero mas") y condicion de STOP.

Todo esto ocurre a una velocidad de 400 kHz (400,000 ciclos de reloj por segundo), asi que una lectura completa tarda microsegundos. Nosotros no vemos nada de esto en nuestro codigo porque la libreria `Wire` de Arduino lo hace todo por nosotros.

En la XIAO, los cables SDA y SCL entre el nRF52840 y el LSM6DS3TR-C estan **dentro de la placa**, son pistas de cobre en el circuito impreso. No tienes que conectar nada.

4. Bluetooth Low Energy (BLE): como los datos viajan al ordenador

4.1. Que es Bluetooth y por que "Low Energy"

Bluetooth es un estandar de comunicacion inalambrica por radio, operando en la banda de **2.4 GHz** (la misma frecuencia que el WiFi). Fue inventado en los años 90 para reemplazar cables

cortos (auriculares, teclados, etc.).

Existen dos "sabores" de Bluetooth:

- **Bluetooth Clasico (BR/EDR)**: Diseñado para transferir mucha informacion continua. Se usa para audio (auriculares), transferencia de archivos, etc. Consume bastante energia.
- **Bluetooth Low Energy (BLE)**: Introducido en Bluetooth 4.0 (2010). Diseñado para dispositivos que envian *poquitos datos de vez en cuando* y necesitan que la bateria dure meses o años. Se usa en pulseras de actividad, sensores, beacons, etc. **Es lo que usamos nosotros.**

BLE no es una "version menor" del Bluetooth clasico. Es un protocolo **completamente diferente**, incompatible con el clasico. Un dispositivo BLE no puede hablar con uno clasico y viceversa (aunque muchos chips modernos, como el nRF52840, soportan ambos).

4.2. Roles en BLE: Peripheral y Central

En BLE siempre hay dos roles:

- **Peripheral (periferico)**: Es el dispositivo "pequeño" que *ofrece datos*. En nuestro caso, la XIAO. El periferico hace dos cosas:
  1. **Advertising**: Antes de que nadie se conecte, el periferico grita al aire "!Estoy aqui! Me llamo XIAO-IMU y tengo datos de sensores!" repetidamente. Esto lo hace emitiendo pequeños paquetes de radio (31 bytes maximo) cada cierto intervalo (tipicamente cada 100 milisegundos). Cualquiera que este escuchando puede ver estos anuncios.
  2. **Servidor GATT**: Una vez que alguien se conecta, el periferico actua como un "servidor" que tiene datos organizados en una estructura (la tabla GATT) para que el central pueda leerlos.
- **Central**: Es el dispositivo "grande" que *consume datos*. En nuestro caso, tu ordenador. El central:
  1. **Escanea**: Escucha los paquetes de advertising para descubrir que perifericos hay cerca.
  2. **Se conecta**: Elige un periferico y establece una conexion.
  3. **Lee datos**: Accede a la tabla GATT del periferico para obtener los valores de los sensores.

Analogia: Piensa en una tienda. El periferico es la tienda que pone un cartel en la puerta (advertising) diciendo "Tenemos frutas". El central es el cliente que ve el cartel, entra (conexion), y mira el catalogo organizado (GATT) para pedir lo que quiere.

5. GATT: la estructura que organiza los datos en BLE

5.1. Que es GATT y por que existe

GATT significa **Generic Attribute Profile**. Es la forma en que BLE organiza los datos que un dispositivo ofrece.

El problema que resuelve GATT: Imagina que tu sensor envia bytes crudos por Bluetooth. Tu ordenador recibe, por ejemplo, 24 bytes. ¿Que significan? ¿Los primeros 12 son el acelerometro y los siguientes 12 el giroscopio? ¿O son 6 valores de 4 bytes cada uno? ¿O es todo un solo string de texto? Sin estructura, es imposible saberlo.

GATT resuelve esto creando una **jerarquia organizada** donde cada dato tiene:

- Un identificador unico (UUID)
- Un tipo y tamaño definido
- Propiedades que dicen que puedes hacer con el (leerlo, escribirlo, recibir actualizaciones, etc.)

5.2. La jerarquia GATT: Perfil, Servicio, Caracteristica

La estructura tiene tres niveles, de mas general a mas especifico:

```
SERVIDOR GATT (todo el dispositivo)
|
+-- SERVICIO "IMU" (UUID: 19B10000-...)
|   |
|   +-- CARACTERISTICA "Acelerometro"
|   |     UUID: 19B10001-...
|   |     Valor: 12 bytes
|   |     Propiedades: READ, NOTIFY
|   |     +-- DESCRIPTOR CCCD (UUID: 0x2902)
|   |           Valor: 0x0000 o 0x0001
|   |
|   +-- CARACTERISTICA "Giroscopio"
|         UUID: 19B10002-...
|         Valor: 12 bytes
|         Propiedades: READ, NOTIFY
|         +-- DESCRIPTOR CCCD (UUID: 0x2902)
|               Valor: 0x0000 o 0x0001
|
+-- SERVICIO "Generic Access" (estandar BLE)
|     (nombre del dispositivo, apariencia, etc.)
|
+-- SERVICIO "Generic Attribute" (estandar BLE)
      (control interno de GATT)
```

Vamos a explicar cada nivel con calma:

5.2.1. Servidor GATT

Es todo el dispositivo. La XIAO entera actua como un servidor GATT. Cuando tu ordenador se conecta, lo primero que hace es pedir "muestrame todo lo que tienes" (esto se llama **service discovery**). El servidor responde con la lista completa de servicios, caracteristicas y descriptores.

5.2.2. Servicio (Service)

Un servicio es un **grupo logico de datos relacionados**. En nuestro caso, creamos un servicio llamado "IMU" que contiene las caracteristicas del acelerometro y el giroscopio. Si tuvieramos tambien un sensor de temperatura, podriamos crear otro servicio "Environment" con una caracteristica de temperatura.

Cada servicio se identifica por un **UUID** (Universally Unique Identifier — "Identificador Unico Universal"). Es un numero de 128 bits (16 bytes) que sirve como nombre unico.

Existen dos tipos de UUIDs:

- **UUIDs estandar (16 bits)**: Definidos oficialmente por el Bluetooth SIG (la organizacion que gestiona el estandar Bluetooth). Son UUIDs cortos que se expanden a 128 bits con una base fija. Ejemplo: `0x180F` = servicio de bateria, `0x180D` = servicio de ritmo cardiaco. Solo los puede asignar el Bluetooth SIG.
- **UUIDs custom (128 bits)**: Los creas tu. Como hay 2^{128} combinaciones posibles (un numero de 39 digitos), la probabilidad de colision es practicamente cero. Se escriben en formato `XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX`. Puedes generar uno con el comando `uuidgen` en la terminal de tu Mac.

Nosotros usamos UUIDs custom porque no existe un servicio estandar para "datos raw de IMU". El UUID que usamos es `19B10000-E8F2-537E-4F6C-D104768A1214`. Lo importante no es el numero en si, sino que sea **el mismo en Arduino y en Python**.

5.2.3. Caracteristica (Characteristic)

Es la **unidad minima de dato**. Cada caracteristica contiene:

1. **UUID**: Para identificarla dentro del servicio. Usamos `19B10001-...` para el acelerometro y `19B10002-...` para el giroscopio.
2. **Valor**: Los bytes de datos propiamente dichos. En nuestro caso, 12 bytes que representan tres numeros float (ax, ay, az o gx, gy, gz).
3. **Propiedades**: Flags (banderas) que definen que operaciones estan permitidas:
   - `READ`: El central puede preguntar "¿cual es tu valor ahora?" y la caracteristica responde con sus bytes. Es como mirar un termometro: tu miras y lees el numero.
   - `WRITE`: El central puede cambiar el valor. Se usa para enviar comandos al periferico (ej: "cambia la frecuencia de muestreo a 50 Hz"). Nosotros no lo usamos.
   - `NOTIFY`: La caracteristica envia su valor **automaticamente** cada vez que cambia, sin que el central lo pida. Es como suscribirte a un canal de noticias: te llegan las actualizaciones solas. **Esto es lo que usamos para streaming.**
   - `INDICATE`: Similar a NOTIFY pero con confirmacion. El central responde "recibido" cada vez. Mas lento pero garantiza entrega. No lo usamos porque la velocidad es mas importante.

5.2.4. Descriptor (Descriptor)

Es metadata *sobre* la caracteristica. El descriptor mas importante es el **CCCD** (Client Characteristic Configuration Descriptor, UUID `0x2902`).

El CCCD es como un "interruptor" de notificaciones. Es un valor de 2 bytes que el central escribe para decir:

- `0x0000`: "No quiero notificaciones" (apagado).
- `0x0001`: "Quiero recibir notificaciones" (encendido).
- `0x0002`: "Quiero recibir indicaciones" (con confirmacion).

Cuando tu llamas a `client.start_notify()` en Python, bleak escribe `0x0001` en el CCCD de esa caracteristica. Cuando llamas a `client.stop_notify()`, escribe `0x0000`.

¿Por que existe el CCCD? Porque seria un desperdicio de energia que el periferico enviara datos constantemente si nadie los quiere. El CCCD permite al central decir "estoy listo para recibir" antes de que el periferico empiece a gastar energia enviando.

5.3. Conexion BLE paso a paso

Cuando tu script Python se conecta a la XIAO, esto es lo que ocurre por debajo:

1. **Scan**: Tu Mac enciende su receptor Bluetooth y escucha paquetes de advertising. Encuentra uno que dice "Me llamo XIAO-IMU".
2. **Connection Request**: Tu Mac envia un paquete `CONNECT_REQ` a la XIAO con los parametros de conexion deseados (intervalo, timeout, etc.).
3. **Connection Established**: La XIAO acepta. Ahora ambos dejan de usar advertising y empiezan a comunicarse en slots de tiempo reservados (esto se llama "connection events").
4. **MTU Exchange**: Negocian el tamaño maximo de paquete. MTU = Maximum Transmission Unit. Por defecto son 23 bytes (20 de payload + 3 de cabecera). Se puede negociar hasta 512 bytes.
5. **Service Discovery**: Tu Mac le pide a la XIAO "dime todos tus servicios y caracteristicas". La XIAO responde con la tabla GATT completa. Esto solo ocurre una vez, al inicio.
6. **Enable Notifications**: Tu Mac escribe `0x0001` en el CCCD de las caracteristicas que le interesan.
7. **Data Streaming**: A partir de ahora, cada vez que la XIAO actualiza el valor de una caracteristica, envia un paquete de notificacion automaticamente.

5.4. Connection interval (intervalo de conexion)

Una vez conectados, el central y el periferico no estan comunicandose todo el tiempo. Eso consumiria demasiada energia. En su lugar, "despiertan" el radio cada cierto intervalo, intercambian datos rapidamente, y vuelven a "dormir".

Este intervalo se llama **connection interval** y se mide en unidades de 1.25 milisegundos. En nuestro codigo Arduino ponemos:

```
BLE.setConnectionInterval(8, 16);
```

Esto significa:

- Minimo: $8 \times 1.25 = 10$ ms
- Maximo: $16 \times 1.25 = 20$ ms

Es decir, cada 10–20 ms hay una oportunidad de enviar datos. Como nuestro sensor muestrea a ∼100 Hz (un dato cada 10 ms), un intervalo de 10 ms nos permite enviar cada muestra en tiempo real.

Nota: Esto es una *solicitud*, no una orden. El sistema operativo de tu Mac puede negociar un intervalo diferente. macOS, por ejemplo, raramente baja de 15 ms.

6. Numeros flotantes y bytes: como viajan los datos

6.1. El problema: BLE solo envía bytes

BLE no sabe que es un "float" ni un "entero". Solo sabe enviar **bytes** (numeros enteros de 0 a 255). Si queremos enviar el numero 1.0234 (un float), necesitamos convertirlo a una secuencia de bytes, enviarlo, y reconvertirlo en el otro lado.

6.2. IEEE 754: como un float se convierte en 4 bytes

Los ordenadores almacenan numeros decimales usando el estandar **IEEE 754**. Un `float` de 32 bits (4 bytes) se descompone asi:

```
Bit:  31   30-23      22-0
      [S] [EEEEEEEE] [MMMMMMMMMMMMMMMMMMMMMMM]
       |   exponente   mantisa (parte fraccionaria)
      signo
```

- **S** (1 bit): 0 = positivo, 1 = negativo.
- **E** (8 bits): Exponente. Determina el "orden de magnitud". Se almacena con un sesgo de 127 (si E = 130, el exponente real es $130 - 127 = 3$).
- **M** (23 bits): Mantisa. La parte fraccionaria. Hay un "1." implicito delante, asi que 23 bits dan 24 bits efectivos de precision (unas 7 cifras decimales).

Ejemplo con el numero 1.0:

- Signo: 0 (positivo)
- Exponente: 127 (es decir, $2^0 = 1$). En binario: 01111111
- Mantisa: 000...0 (porque $1.0 = 1.0 \times 2^0$, y la parte fraccionaria es cero)

Resultado en binario: `0 01111111 00000000000000000000000`

En hexadecimal: `0x3F800000`

En bytes: `[0x3F, 0x80, 0x00, 0x00]`

6.3. Little-endian y Big-endian

Los 4 bytes de un float se pueden ordenar de dos formas:

- **Big-endian**: El byte mas significativo va primero. El numero `0x3F800000` se almacena como `[3F, 80, 00, 00]`.
- **Little-endian**: El byte menos significativo va primero. `[00, 00, 80, 3F]`.

El procesador ARM Cortex-M4 (que esta dentro del nRF52840) usa **little-endian**. Los procesadores Intel/AMD y Apple Silicon de tu Mac tambien. Esto es conveniente: los bytes llegan en el mismo orden en que fueron escritos, sin necesidad de reordenar.

En Python, indicamos el orden de bytes en el formato de `struct.unpack`:

- `'<'` = little-endian
- `'>'` = big-endian
- `'f'` = un float de 32 bits (4 bytes)
- `'<fff'` = tres floats little-endian = 12 bytes

6.4. memcpy: copiar bytes crudos en Arduino

En el codigo Arduino, usamos `memcpy` para "empaquetar" los floats en un array de bytes:

```
float ax = 1.0;            // 4 bytes en memoria
uint8_t buffer[12];         // 12 bytes vacios
memcpy(&buffer[0], &ax, 4); // copia los 4 bytes de ax al buffer
```

`memcpy` significa "memory copy" (copia de memoria). Sus parametros son:

1. **destino**: `&buffer[0]` — la direccion de memoria donde queremos copiar (posicion 0 del array).
2. **origen**: `&ax` — la direccion de memoria donde esta el float. El `&` significa "la direccion de".
3. **cantidad**: `4` — cuantos bytes copiar (un float = 4 bytes).

`memcpy` no sabe ni le importa que hay en esos bytes. Solo copia bytes de un sitio a otro. Esto es lo que necesitamos: tomar los 4 bytes que representan el float en memoria y ponerlos en nuestro buffer de envio BLE.

7. Las librerias: quien hizo que y como funciona por dentro

7.1. Lado Arduino (firmware de la XIAO)

7.1.1. ArduinoBLE

- **Proveedor**: Arduino (empresa/fundacion).
- **Repositorio**: https://github.com/arduino-libraries/ArduinoBLE
- **Que hace**: Proporciona toda la funcionalidad BLE en Arduino. Te permite crear servicios, caracteristicas, hacer advertising, gestionar conexiones, etc.
- **Como funciona por dentro**: ArduinoBLE es una capa de abstraccion sobre el **SoftDevice** de Nordic Semiconductor. El SoftDevice es un firmware propietario (un blob binario, no puedes ver su codigo fuente) que Nordic instala en parte de la memoria flash del nRF52840. Este firmware controla directamente el hardware de radio (el transceptor de 2.4 GHz, el sintetizador de frecuencia, etc.). ArduinoBLE llama a las funciones del SoftDevice a traves de la capa mbed-OS.

  La cadena completa es:

  ```
  Tu codigo Arduino
    -> ArduinoBLE (API amigable)
      -> mbed-OS cordio (capa de abstraccion)
        -> SoftDevice de Nordic (firmware del radio)
          -> Hardware de radio del nRF52840
  ```

7.1.2. Seeed_Arduino_LSM6DS3

- **Proveedor**: Seeed Studio (la empresa que fabrica la XIAO).
- **Repositorio**: https://github.com/Seeed-Studio/Seeed_Arduino_LSM6DS3
- **Que hace**: Permite leer datos del sensor LSM6DS3TR-C de forma sencilla con funciones como `readFloatAccelX()`.
- **Como funciona por dentro**: Es un wrapper (envoltorio) sobre el driver oficial de STMicroelectronics. Internamente usa la libreria Wire para comunicarse por I^2C:

  ```
  myIMU.readFloatAccelX()
    -> LSM6DS3::readRegisterInt16(OUTX_L_XL)
      -> Wire.beginTransmission(0x6A)
      -> Wire.write(0x28)     // registro a leer
      -> Wire.endTransmission()
      -> Wire.requestFrom(0x6A, 2)  // pedir 2 bytes
  ```

```
      -> byte_low = Wire.read()
      -> byte_high = Wire.read()
    -> raw_value = (byte_high << 8) | byte_low
    -> return raw_value * 0.000061  // convertir a g
```

7.1.3. Wire

- **Proveedor**: Arduino (viene incluida con Arduino IDE, es una "core library").
- **Que hace**: Implementa el protocolo I^2C.
- **Como funciona por dentro**: Configura los registros de hardware del modulo I^2C del nRF52840 (llamado TWIM — Two-Wire Interface Master). Gestiona el timing de SCL, los ACK/NACK, y las interrupciones.

7.2. Lado Python (tu ordenador)

7.2.1. bleak

- **Proveedor**: Henrik Blidh (desarrollador independiente, proyecto open source).
- **Repositorio**: https://github.com/hbldh/bleak
- **PyPI**: https://pypi.org/project/bleak/
- **Que hace**: Permite a Python comunicarse con dispositivos BLE. Es la libreria estandar de facto para BLE en Python.
- **Como funciona por dentro**: bleak no controla el Bluetooth directamente. Se apoya en las APIs del sistema operativo:
  - **macOS**: Usa `CoreBluetooth`, el framework de Apple para BLE. Se comunica con el a traves de `pyobjc` (un bridge que permite llamar a APIs de Objective-C desde Python). Cuando llamas a `BleakScanner.discover()`, bleak le dice a CoreBluetooth "escanea dispositivos BLE", y CoreBluetooth le pide al chip Bluetooth del Mac que encienda el receptor y escuche paquetes de advertising.
  - **Linux**: Usa `BlueZ` (el stack Bluetooth del kernel Linux) a traves de D-Bus (un sistema de comunicacion entre procesos).
  - **Windows**: Usa `WinRT` (Windows Runtime APIs) para acceder al Bluetooth.

  La cadena en macOS es:

  ```
  Tu codigo Python
    -> bleak (API asíncrona en Python)
      -> pyobjc (bridge Python-Objective-C)
        -> CoreBluetooth (framework de Apple)
          -> Driver Bluetooth de macOS
            -> Chip Bluetooth del Mac
  ```

  Toda la API de bleak es asincrona. Esto significa que usa `asyncio` y que todas las operaciones importantes usan `async/await`. ¿Por que? Porque las operaciones BLE (escanear, conectar, esperar notificaciones) pueden tardar un tiempo indefinido, y no queremos que el programa se "congele" mientras espera.

7.2.2. struct (biblioteca estandar de Python)

- **Proveedor**: Python Software Foundation (viene incluido con Python, no hay que instalar nada).
- **Que hace**: Convierte entre valores de Python (int, float, etc.) y secuencias de bytes en formato C.
- **Funcion que usamos**: `struct.unpack(formato, bytes)`

  Toma una secuencia de bytes y la descompone en valores de Python segun el formato especificado. El formato `'<fff'` significa:

  - `<`: Los bytes estan en orden little-endian.
  - `f`: Un float de 32 bits (4 bytes).
  - Tres `f`: Tres floats = 12 bytes total.

  Devuelve una **tupla** (tipo de dato inmutable de Python) con los tres valores: `(1.0234, -0.0012, 0.9876)`

7.2.3. asyncio (biblioteca estandar de Python)

- **Proveedor**: Python Software Foundation (incluido con Python 3.4+).
- **Que hace**: Permite escribir codigo asíncrono (que puede "esperar" sin bloquear el programa).
- **Conceptos clave**:
  - **Event loop** (bucle de eventos): Es el "corazon" de asyncio. Es un bucle infinito que revisa constantemente: "¿hay algun evento nuevo? ¿alguna operacion termino? ¿llego algun dato?". Cuando detecta algo, ejecuta el codigo correspondiente.

    Analogia: imagina un camarero en un restaurante. En vez de quedarse parado esperando a que la cocina termine un plato, va a tomar nota de otras mesas. Cuando la cocina avisa que un plato esta listo, el camarero lo lleva. El event loop es ese camarero.
  - **Coroutine**: Una funcion definida con `async def`. A diferencia de una funcion normal, una coroutine puede "pausarse" (con `await`) y reanudarse mas tarde. Cuando se pausa, el event loop puede ejecutar otras coroutines.
  - **await**: Pausa la coroutine actual hasta que la operacion termine. Durante la pausa, el event loop puede hacer otras cosas (como procesar notificaciones BLE entrantes).
  - **asyncio.run(coroutine)**: Crea un event loop, ejecuta la coroutine, y limpia todo al terminar. Es el punto de entrada para cualquier programa asincrono.
  - **asyncio.sleep(segundos)**: Pausa la coroutine durante N segundos **sin bloquear el event loop**. Si usaras `time.sleep()` en su lugar, *bloquearias* el event loop y no se procesarian las notificaciones BLE mientras dure la pausa.

Parte II

El codigo explicado linea por linea

8. Codigo Arduino completo

8.1. Linea 1: `#include <ArduinoBLE.h>`

```
#include <ArduinoBLE.h>
```

`#include` es una directiva del preprocesador de C++. Antes de compilar tu codigo, el preprocesador reemplaza esta linea por **todo el contenido** del archivo `ArduinoBLE.h`. Es como hacer copy-paste automatico de ese archivo.

Los angulos `< >` indican que el archivo esta en las carpetas del sistema (librerias instaladas). Si usaramos comillas `" "`, buscaria en la carpeta del proyecto.

Al incluir `ArduinoBLE.h`, traemos al scope (ambito) las siguientes clases y objetos:

- `BLE`: Un objeto **singleton** global (singleton = solo existe uno en todo el programa). Controla el radio BLE del chip. Con el inicializas BLE, pones nombre, empiezas advertising, etc.
- `BLEService`: Clase para crear servicios GATT. Un servicio es un contenedor de caracteristicas (como explicamos antes).
- `BLECharacteristic`: Clase para crear caracteristicas GATT. Cada caracteristica almacena un valor (array de bytes) y tiene propiedades (READ, NOTIFY, etc.).
- `BLEDevice`: Clase que representa un dispositivo BLE conectado. Cuando un central se conecta a tu XIAO, obtienes un objeto `BLEDevice` que representa a ese central.

8.2. Linea 2: `#include "LSM6DS3.h"`

```
#include "LSM6DS3.h"
```

Incluye la libreria del sensor IMU. Trae la clase `LSM6DS3` que abstrae toda la comunicacion I^2C con el chip sensor. Sin esta libreria, tendriamos que escribir manualmente los comandos I^2C para leer cada registro (decenas de lineas de codigo).

Las comillas `" "` en vez de `< >` son una convencion: indican que es una libreria externa (no del core de Arduino), aunque en la practica ambas sintaxis funcionan igual.

8.3. Linea 3: `#include "Wire.h"`

```
#include "Wire.h"
```

Incluye la libreria Wire que implementa I^2C. Nosotros no la usamos directamente en nuestro codigo, pero la libreria LSM6DS3 la necesita internamente. Sin este include, el compilador daria un error diciendo que no encuentra las funciones de Wire.

8.4. Linea 4: Crear el objeto IMU

```
LSM6DS3 myIMU(I2C_MODE, 0x6A);
```

Crea una **instancia** (un objeto concreto) de la clase `LSM6DS3`. Le damos el nombre `myIMU`. Es como decir "crea un sensor de tipo LSM6DS3 y llamalo myIMU".

El **constructor** (la funcion que se ejecuta al crear el objeto) recibe dos parametros:

1. `I2C_MODE`: Una constante definida en la libreria. Le dice al objeto que se comunique por I^2C (la alternativa seria `SPI_MODE`, otro protocolo de comunicacion, pero la XIAO usa I^2C).
2. `0x6A`: La direccion I^2C del sensor. El `0x` indica que es un numero en hexadecimal. `6A` en hexadecimal = 106 en decimal. Esta direccion viene predeterminada de fabrica en el chip.

Importante: Crear el objeto no inicializa el sensor. Solo guarda los parametros en memoria. La inicializacion real ocurre cuando llamas a `myIMU.begin()`.

Internamente, el constructor hace algo como:

```
// Pseudocodigo interno del constructor
this->commMode = I2C_MODE;
this->I2CAddress = 0x6A;
this->accelRange = 2;      // default +/- 2g
this->gyroRange = 245;     // default +/- 245 dps
this->accelODR = 416;      // default 416 Hz
this->gyroODR = 416;       // default 416 Hz
```

8.5. Lineas 5–7: Crear servicio y caracteristicas BLE

```
BLEService imuService("19B10000-E8F2-537E-4F6C-D104768A1214");
```

Crea un servicio GATT. El parametro es el **UUID del servicio** como string. Este UUID es un identificador unico que permite a los clientes (tu Python) encontrar este servicio especifico entre todos los servicios del dispositivo.

```
BLECharacteristic accelChar(
    "19B10001-E8F2-537E-4F6C-D104768A1214",
    BLERead | BLENotify,
    12
);
```

Crea una caracteristica GATT. Tres parametros:

1. **UUID** (`"19B10001-..."`): Fijate que cambia el `0001` respecto al servicio (`0000`). Esto es una convencion para mantener los UUIDs relacionados.
2. **Propiedades** (`BLERead | BLENotify`): `BLERead` y `BLENotify` son constantes numericas (por ejemplo, `BLERead = 0x02` y `BLENotify = 0x10`). El operador `|` (OR de bits) las combina: $0x02|0x10 = 0x12$. Esto crea un unico byte donde los bits de READ y NOTIFY estan activados.
3. **Tamaño** (`12`): Tamaño maximo del valor en bytes. Necesitamos 12 porque enviamos 3 floats: $3 \times 4 = 12$ bytes.

La caracteristica del giroscopio es identica pero con UUID `0002`:

```
BLECharacteristic gyroChar(
    "19B10002-E8F2-537E-4F6C-D104768A1214",
    BLERead | BLENotify,
    12
);
```

8.6. Funcion setup(): inicializacion

```
void setup() {
```

`setup()` es una funcion especial de Arduino que se ejecuta **una sola vez** al encender la placa. Aqui va toda la inicializacion. `void` significa que la funcion no devuelve ningun valor.

```
  Serial.begin(115200);
```

Inicializa la comunicacion serie por USB. `Serial` es un objeto global que representa el puerto USB de la XIAO. Cuando la XIAO esta conectada por USB al ordenador, puedes ver mensajes de debug en el "Monitor Serie" del Arduino IDE.

`115200` es la velocidad en **baudios** (bits por segundo). Es una velocidad estandar rapida. Tanto el emisor (XIAO) como el receptor (tu ordenador) deben usar la misma velocidad.

```
  if (myIMU.begin() != 0) {
    Serial.println("IMU error");
    while (1);
  }
```

`myIMU.begin()` inicializa el sensor. Internamente hace:

1. Llama a `Wire.begin()` para inicializar el bus I^2C (configura los pines SDA/SCL, activa el modulo de hardware I^2C del nRF52840).
2. Lee el registro `WHO_AM_I` (direccion `0x0F`) del sensor. Este registro siempre contiene el valor `0x6A`. Es como preguntarle al sensor "¿quien eres?" para confirmar que esta ahi y funciona.
3. Escribe en los registros de configuracion del sensor para establecer los rangos, frecuencias de muestreo, y filtros por defecto.
4. Devuelve `0` si todo fue bien, o un numero distinto de cero si algo fallo (por ejemplo, si el sensor no respondio al `WHO_AM_I`).

Si falla (`!= 0`), imprime un mensaje de error y entra en `while(1)`: un bucle infinito que detiene el programa. Esto es un patron comun en embedded: si algo critico falla en la inicializacion, es mejor parar que seguir con un sistema roto.

```
  if (!BLE.begin()) {
    Serial.println("BLE failed!");
    while (1);
  }
```

`BLE.begin()` inicializa el stack BLE. Internamente:

1. Inicializa el SoftDevice de Nordic (el firmware propietario que controla el radio de 2.4 GHz).
2. Reserva memoria RAM para las estructuras BLE (tabla GATT, buffers de conexion, colas de paquetes).
3. Configura las interrupciones de hardware para que el radio pueda avisar al procesador cuando llega un paquete.
4. Devuelve `true` si todo fue bien, `false` si fallo.

El `!` (NOT) niega el valor: si `begin()` devuelve `false` (fallo), `!false = true`, y entramos al bloque de error.

```
  BLE.setLocalName("XIAO-IMU");
```

Establece el nombre que aparecera en los paquetes de advertising. Cuando hagas un scan BLE desde tu Mac o telefono, veras "XIAO-IMU" en la lista. Este nombre se incluye en el paquete de advertising como un campo de tipo "Complete Local Name". Limite: 29 caracteres.

```
  BLE.setAdvertisedService(imuService);
```

Le dice al stack BLE que incluya el UUID de nuestro servicio IMU en los paquetes de advertising. Esto permite que los centrales puedan filtrar por servicio antes de conectarse. Sin esta linea, el dispositivo se anunciaria sin decir que servicios tiene, y el central tendria que conectarse "a ciegas" para descubrirlo.

```
  imuService.addCharacteristic(accelChar);
  imuService.addCharacteristic(gyroChar);
```

Asocia las caracteristicas al servicio. Construye la jerarquia GATT. Internamente, el servicio mantiene una lista enlazada de caracteristicas. Al añadir una, se agrega al final de esa lista.

```
  BLE.addService(imuService);
```

Registra el servicio completo (con todas sus caracteristicas) en la tabla GATT del stack BLE. A partir de ahora, el servicio "existe" y sera visible para cualquier central que se conecte y haga service discovery.

```
  BLE.setConnectionInterval(8, 16);
```

Ya lo explicamos en la seccion de connection interval. Pedimos un intervalo de 10–20 ms.

```
  BLE.advertise();
```

Empieza a emitir paquetes de advertising. A partir de este momento, la XIAO es visible para cualquier scanner BLE. Los paquetes se emiten aproximadamente cada 100 ms hasta que alguien se conecte.

8.7. Funcion loop(): el bucle principal

```
void loop() {
  BLEDevice central = BLE.central();
```

`loop()` se ejecuta en bucle infinito despues de `setup()`. Es el "main loop" del programa.

`BLE.central()` comprueba si hay un dispositivo central conectado. Devuelve un objeto `BLEDevice`:

- Si hay conexion: devuelve un objeto valido que contiene la direccion MAC del central.
- Si no hay conexion: devuelve un objeto "vacio" que evalua a `false` en un `if`.

```
  if (central) {
    Serial.println("Connected");
    while (central.connected()) {
```

Si hay un central conectado, entramos en un bucle `while` que continua mientras la conexion este activa. `central.connected()` devuelve `false` cuando:

- El central se desconecta voluntariamente.
- Se pierde la conexion (la XIAO se aleja demasiado, hay interferencia, etc.).
- Pasan mas de 4 segundos sin comunicacion (supervision timeout).

```
float ax = myIMU.readFloatAccelX();
float ay = myIMU.readFloatAccelY();
float az = myIMU.readFloatAccelZ();
float gx = myIMU.readFloatGyroX();
float gy = myIMU.readFloatGyroY();
float gz = myIMU.readFloatGyroZ();
```

Lee los 6 valores del IMU. Cada llamada a `readFloat...()`:

1. Envia un comando I^{2}C al sensor pidiendo 2 bytes del registro correspondiente.
2. Recibe los 2 bytes (un entero de 16 bits con signo).
3. Multiplica por el factor de escala para convertir a unidades fisicas.
4. Devuelve un `float` (numero decimal de 32 bits).

El acelerometro devuelve valores en **g** (1g $\approx$ 9.81 m/s^2). En reposo: $a_x \approx 0, a_y \approx 0, a_z \approx 1.0$.

El giroscopio devuelve valores en **grados/segundo**. En reposo: $g_x \approx 0, g_y \approx 0, g_z \approx 0$.

```
uint8_t accelBuf[12];
memcpy(&accelBuf[0], &ax, 4);
memcpy(&accelBuf[4], &ay, 4);
memcpy(&accelBuf[8], &az, 4);
accelChar.writeValue(accelBuf, 12);
```

`uint8_t accelBuf[12]`: Declara un array de 12 bytes en la pila (stack). `uint8_t` = unsigned integer de 8 bits = un byte (valor de 0 a 255).

Las tres lineas de `memcpy` copian los bytes crudos de cada float al buffer. Despues de ejecutarse, el buffer contiene:

```
Posicion:  0  1  2  3    4  5  6  7    8  9 10 11
Contenido: [--- ax ---]  [--- ay ---]  [--- az ---]
           4 bytes       4 bytes       4 bytes
```

`accelChar.writeValue(accelBuf, 12)`: Actualiza el valor de la caracteristica GATT. Internamente:

1. Copia los 12 bytes al buffer interno de la caracteristica.
2. Si un central esta suscrito a notificaciones (escribio 0x0001 en el CCCD), encola un paquete de notificacion BLE.
3. El paquete se transmitira por radio en el siguiente connection event (el proximo "despertar" del intervalo de conexion).

El bloque del giroscopio es identico:

```
uint8_t gyroBuf[12];
memcpy(&gyroBuf[0], &gx, 4);
memcpy(&gyroBuf[4], &gy, 4);
memcpy(&gyroBuf[8], &gz, 4);
gyroChar.writeValue(gyroBuf, 12);
```

```
delay(10);
```

Pausa la ejecucion 10 milisegundos. Esto limita la frecuencia de muestreo a aproximadamente 1/0.010 = 100 Hz.

¿Por que "aproximadamente"? Porque las lecturas I^2C (6 transacciones) y los envios BLE (2 `writeValue`) tambien tardan tiempo (1–3 ms en total). Asi que el ciclo real es de ∼12 ms, dando ∼83 Hz. Para la mayoria de aplicaciones de IMU esto es suficiente.

```
    }
    Serial.println("Disconnected");
  }
}
```

Cuando el central se desconecta, sale del `while` e imprime "Disconnected". El `loop()` vuelve a empezar: llama de nuevo a `BLE.central()`, que no encuentra conexion, asi que no entra al `if`, y el loop se repite. El advertising se reanuda automaticamente cuando no hay conexion activa, asi que la XIAO vuelve a ser visible para nuevas conexiones.

9. Codigo Python completo

9.1. Imports

```
import asyncio
```

Importa el modulo `asyncio`. Ya explicamos a fondo que es asyncio en la seccion de librerias. En resumen: nos permite escribir codigo que "espera" (conexion BLE, datos, etc.) sin congelar el programa.

```
import struct
```

Importa el modulo `struct`. Lo usamos para convertir los bytes que llegan por BLE de vuelta a numeros float.

```
import time
```

Importa el modulo `time`. Lo usamos para obtener timestamps (`time.time()` devuelve los segundos desde el 1 de enero de 1970, con precision de microsegundos).

```
from bleak import BleakClient, BleakScanner
```

Importa dos clases de bleak:

- `BleakScanner`: Para escanear dispositivos BLE cercanos. Es como "mirar que hay en el aire".
- `BleakClient`: Para conectarse a un dispositivo especifico y comunicarse con el (leer/escribir caracteristicas, suscribirse a notificaciones, etc.).

9.2. Constantes

```
ACCEL_UUID = "19b10001-e8f2-537e-4f6c-d104768a1214"
GYRO_UUID  = "19b10002-e8f2-537e-4f6c-d104768a1214"
```

Los UUIDs de las caracteristicas GATT. **Deben coincidir exactamente** con los del sketch Arduino. Estan en minusculas porque bleak normaliza todos los UUIDs a minusculas internamente.

Estos strings son la "direccion" que Python usara para decirle a bleak "quiero los datos de esta caracteristica especifica".

9.3. Variables globales

```
latest_accel = (0, 0, 0)
latest_gyro = (0, 0, 0)
```

Tuplas que almacenan la ultima lectura recibida. Las inicializamos en $(0, 0, 0)$. Se actualizan desde los callbacks cada vez que llega una notificacion.

¿Que es una tupla? Es un tipo de dato de Python similar a una lista, pero inmutable (no se puede modificar una vez creada). Se escribe con parentesis: `(1, 2, 3)`. Usamos tuplas porque `struct.unpack` devuelve tuplas.

¿Por que `global`? En Python, si quieres modificar una variable definida fuera de una funcion desde dentro de esa funcion, debes declararla como `global`. Sin esa declaracion, Python crearia una variable local con el mismo nombre (y la global no se modificaria).

9.4. Callbacks (funciones que se llaman automaticamente)

```
def accel_handler(_, data: bytearray):
    global latest_accel
    latest_accel = struct.unpack('<fff', data)
    ax, ay, az = latest_accel
    print(f"Accel: x={ax:.4f}  y={ay:.4f}  z={az:.4f} g")
```

Esta funcion se ejecuta **automaticamente** cada vez que la XIAO envia una notificacion de la caracteristica del acelerometro. Nosotros nunca la llamamos directamente; la registramos con `start_notify` y bleak la llama cuando llegan datos.

Parametros:

- `_`: Es el objeto `BleakGATTCharacteristic` que origino la notificacion. Lo nombramos `_` porque no lo necesitamos. En Python, `_` es una convencion para "variable que no me importa".
- `data: bytearray`: Los bytes crudos que llegaron por BLE. `bytearray` es un tipo de Python: una secuencia mutable de bytes. Contendra exactamente 12 bytes (los 3 floats que empaquetamos en Arduino). El `: bytearray` es un "type hint" (anotacion de tipo): no cambia el comportamiento del codigo, solo documenta que tipo de dato se espera.

Cuerpo:

- `global latest_accel`: Declara que vamos a modificar la variable global.
- `struct.unpack('<fff', data)`: Toma los 12 bytes de `data` y los desempaqueta como 3 floats little-endian. Devuelve una tupla, por ejemplo `(0.0234, -0.0012, 0.9876)`.
- `ax, ay, az = latest_accel`: "Destructuring" o "unpacking". Asigna cada elemento de la tupla a una variable individual. Es equivalente a:

  ```
  ax = latest_accel[0]
  ay = latest_accel[1]
  az = latest_accel[2]
  ```

- `print(f"...")`: "f-string" (formatted string literal). Las expresiones entre `{}` se evaluan y se insertan en el string. `:.4f` formatea el float con 4 decimales.

El handler del giroscopio es identico pero con `:.2f` (2 decimales):

```
def gyro_handler(_, data: bytearray):
    global latest_gyro
    latest_gyro = struct.unpack('<fff', data)
    gx, gy, gz = latest_gyro
    print(f"Gyro:  x={gx:.2f}  y={gy:.2f}  z={gz:.2f} deg/s")
```

¿Como llega la notificacion al callback? La cadena es:

1. La XIAO actualiza el valor de la caracteristica y el radio BLE envia un paquete de notificacion.
2. El chip Bluetooth de tu Mac recibe el paquete.
3. CoreBluetooth (macOS) procesa el paquete y notifica a bleak a traves de un "delegate" de Objective-C.
4. bleak pone el evento en la cola del event loop de asyncio.
5. El event loop ejecuta tu callback con los datos.

Todo esto ocurre en milisegundos.

9.5. Funcion principal

```
async def main():
```

Define `main` como una **coroutine asincrona**. `async def` es como `def` pero indica que la funcion puede usar `await` y puede pausarse/reanudarse.

```
    print("Scanning...")
    device = await BleakScanner.find_device_by_name(
        "XIAO-IMU", timeout=10.0
    )
```

Escanea dispositivos BLE buscando uno con nombre "XIAO-IMU".

`BleakScanner.find_device_by_name()` es un metodo **de clase** (se llama en la clase, no en una instancia). Es asincrono, por eso usamos `await`.

Internamente:

1. Crea un objeto `BleakScanner`.
2. Llama a CoreBluetooth para empezar a escanear.
3. CoreBluetooth activa el receptor Bluetooth y empieza a escuchar paquetes de advertising.
4. Por cada paquete recibido, compara el campo "Local Name" con `"XIAO-IMU"`.
5. Si encuentra coincidencia, para el scan y devuelve un objeto `BLEDevice` con la informacion del dispositivo.
6. Si pasan 10 segundos sin encontrarlo, devuelve `None`.

El `await` pausa la coroutine `main()` mientras el scan esta activo. El event loop queda libre para procesar otros eventos (aunque en este caso no hay otros).

```
    if not device:
        print("XIAO-IMU not found!")
        return
```

Si `device` es `None` (no se encontro), muestra un mensaje y termina la funcion. `return` en una coroutine asincrona la finaliza y devuelve `None`.

```
    print(f"Found: {device.address}")
```

Imprime la direccion del dispositivo. En macOS, la "direccion" no es una MAC real (Apple la oculta por privacidad) sino un UUID de CoreBluetooth. En Linux si veras la MAC real (algo como `A4:CF:12:8B:3E:D2`).

```
    async with BleakClient(device, timeout=20.0) as client:
```

Esto hace varias cosas a la vez:

1. `BleakClient(device, timeout=20.0)`: Crea un objeto cliente BLE configurado para conectarse al dispositivo encontrado. `timeout=20.0` es el tiempo maximo en segundos para establecer la conexion.
2. `async with ... as client`: Es un "context manager asincrono". Es un patron de Python que garantiza limpieza de recursos. Equivale a:

   ```
   client = BleakClient(device, timeout=20.0)
   await client.connect()    # se conecta
   try:
       # ... tu codigo ...
   finally:
       await client.disconnect()  # SIEMPRE desconecta
   ```

   La ventaja del `async with` es que la desconexion ocurre **siempre**, incluso si hay un error o excepcion dentro del bloque. Asi no dejas conexiones BLE abiertas "colgadas".

Al entrar al bloque, bleak:

1. Pide a CoreBluetooth que establezca conexion con el periferico.
2. Espera la confirmacion de conexion.
3. Ejecuta **service discovery**: le pide al periferico su tabla GATT completa y la almacena en `client.services`.

```
        print(f"Connected: {client.is_connected}")
```

`client.is_connected` es una propiedad (no una funcion, por eso no lleva parentesis) que devuelve `True` si hay una conexion BLE activa.

```
        await client.start_notify(ACCEL_UUID, accel_handler)
        await client.start_notify(GYRO_UUID, gyro_handler)
```

Esta es la linea mas importante de todo el programa Python.

`start_notify(uuid, callback)` hace:

1. Busca la caracteristica por UUID en la tabla GATT que se obtuvo durante service discovery.
2. Verifica que la caracteristica tenga la propiedad NOTIFY (si no, lanza una excepcion).
3. Escribe `0x0001` (2 bytes, little-endian) en el **CCCD** (descriptor `0x2902`) de esa caracteristica. Esto le dice a la XIAO: "activa las notificaciones para esta caracteristica".
4. Registra `callback` (nuestra funcion `accel_handler`) en un diccionario interno de bleak, indexado por el "handle" (identificador numerico interno) de la caracteristica.

5. A partir de ahora, cada vez que la XIAO haga `writeValue()` en esa caracteristica, el stack BLE enviara los datos y bleak llamara a nuestro callback.

```
        print("Streaming... Ctrl+C to stop\n")
```

Mensaje informativo. `\n` es un salto de linea extra.

```
        try:
            while True:
                await asyncio.sleep(1.0)
        except KeyboardInterrupt:
            pass
```

Este bloque mantiene el programa vivo mientras recibe datos:

- `while True`: Bucle infinito. Sin esto, la funcion terminaria inmediatamente y se cerraria la conexion BLE.
- `await asyncio.sleep(1.0)`: Pausa la coroutine 1 segundo. **Importantisimo**: usamos `asyncio.sleep` (asincrono) y NO `time.sleep` (sincrono). La diferencia:
  - `asyncio.sleep`: Pausa *solo esta coroutine*. El event loop sigue activo y puede procesar notificaciones BLE. Los callbacks se ejecutan normalmente durante la pausa.
  - `time.sleep`: Bloquea *todo el hilo*, incluyendo el event loop. Ninguna notificacion BLE se procesaria durante la pausa. Los datos se perderian.
- `except KeyboardInterrupt`: Captura la excepcion que se genera cuando presionas Ctrl+C en la terminal. `pass` significa "no hacer nada especial, solo salir del bucle limpiamente". Sin este try/except, al pulsar Ctrl+C verias un traceback de error feo.

```
        await client.stop_notify(ACCEL_UUID)
        await client.stop_notify(GYRO_UUID)
```

Cancela las suscripciones. Internamente:

1. Escribe `0x0000` en el CCCD de cada caracteristica ("apaga notificaciones").
2. Elimina los callbacks del diccionario interno de bleak.

No es estrictamente necesario porque `disconnect()` tambien limpia todo, pero es buena practica hacer un cierre ordenado.

```
    print("Disconnected.")
```

Al salir del `async with`, bleak llama automaticamente a `client.disconnect()`, que:

1. Envia un paquete de terminacion de conexion BLE.
2. Libera los recursos del radio Bluetooth en el SO.
3. La XIAO detecta la desconexion y sale del `while(central.connected())`.

9.6. Punto de entrada

```
if __name__ == "__main__":
    asyncio.run(main())
```

- `if __name__ == "__main__"`: Comprueba si este archivo se esta ejecutando directamente (no importado como modulo). `__name__` es una variable especial que Python establece automaticamente:
  - Si ejecutas `python script.py`: `__name__ = "__main__"`
  - Si haces `import script`: `__name__ = "script"`
- `asyncio.run(main())`: Punto de entrada del programa asincrono. Hace tres cosas:
  1. Crea un nuevo event loop.
  2. Ejecuta la coroutine `main()` hasta que termine.
  3. Cierra el event loop y limpia todos los recursos.

  Nota para Jupyter Notebook: En un notebook, ya hay un event loop ejecutandose (el del propio notebook). Llamar a `asyncio.run()` dentro de un event loop existente da error. En su lugar, usa directamente:

```
1 await main()
```

Parte III

Apendices

10. Extension: guardar datos en CSV

Para alimentar tus notebooks de Madgwick con datos BLE:

```
1  import csv
2
3  # Abrir archivo CSV al inicio
4  csv_file = open('data/ble_capture.csv', 'w', newline='')
5  writer = csv.writer(csv_file)
6  writer.writerow([
7      'timestamp',
8      'accel_x', 'accel_y', 'accel_z',
9      'gyro_x', 'gyro_y', 'gyro_z'
10 ])
11
12 def accel_handler(_, data: bytearray):
13     global latest_accel
14     latest_accel = struct.unpack('<fff', data)
15
16 def gyro_handler(_, data: bytearray):
17     global latest_gyro
18     latest_gyro = struct.unpack('<fff', data)
19     # Escribir al CSV cuando llega el gyro
20     t = time.time()
21     ax, ay, az = latest_accel
22     gx, gy, gz = latest_gyro
23     writer.writerow([t, ax, ay, az, gx, gy, gz])
```

11. Diagnostico y solucion de problemas

11.1. El dispositivo no aparece en el scan

- Verifica que el sketch Arduino se cargo correctamente (abre el Monitor Serie y busca "Advertising as XIAO-IMU").
- Asegura que no hay otro dispositivo ya conectado a la XIAO (BLE solo permite una conexion simultanea por defecto).
- En macOS: Preferencias del Sistema > Bluetooth > verificar que esta activado.
- El nombre es case-sensitive: "XIAO-IMU" $\neq$ "xiao-imu".
- Aumenta el timeout del scan a 15–20 segundos.

11.2. Se conecta pero no llegan datos

- Verifica que los UUIDs en Python son **identicos** a los del Arduino (copia y pega, no escribas a mano).
- Verifica que `myIMU.begin()` devolvio 0 (Monitor Serie).
- Ejecuta un script que liste servicios y caracteristicas para ver que expone realmente el dispositivo.

11.3. Los valores no tienen sentido

- Verifica que usas `'<fff'` (little-endian) y no `'>fff'` (big-endian).
- Verifica que `len(data) == 12`. Si es diferente, el empaquetado no coincide.
- En reposo, el acelerometro debe leer $\approx (0, 0, 1)$ g y el giroscopio $\approx (0, 0, 0)$ dps.

12. Referencias

- Bleak documentation: https://bleak.readthedocs.io/
- Bleak GitHub: https://github.com/hbldh/bleak
- ArduinoBLE reference: https://www.arduino.cc/reference/en/libraries/arduinoble/
- Seeed XIAO BLE Sense wiki — IMU: https://wiki.seeedstudio.com/XIAO-BLE-Sense-IMU-Usage/
- Seeed XIAO BLE Sense wiki — Bluetooth: https://wiki.seeedstudio.com/XIAO-BLE-Sense-Bluetooth
- LSM6DS3 library: https://github.com/Seeed-Studio/Seeed_Arduino_LSM6DS3
- Bluetooth GATT specification: https://www.bluetooth.com/specifications/gatt/
- IEEE 754 floating point: https://en.wikipedia.org/wiki/IEEE_754
- Python struct module: https://docs.python.org/3/library/struct.html
- Python asyncio: https://docs.python.org/3/library/asyncio.html